

Índice (←)

Código Fuente (.c, .cpp, .h)

Instrucciones de alto nivel escritas por el programador.

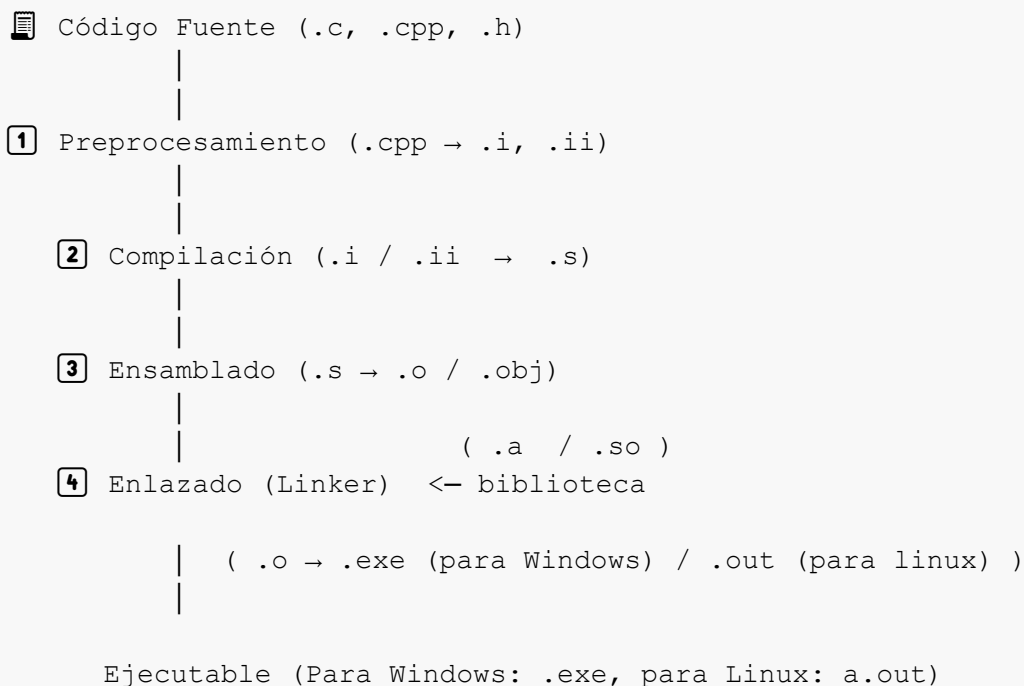
Código de Máquina (.exe, .out)

Instrucciones binarias (0 y 1) que entiende el procesador directamente.

Código Objeto (.o, .obj)

Paso intermedio en lenguaje máquina, pero sin enlazar las bibliotecas. (binario, contiene código máquina + metadata)

Etapas del Compilador



1 Preprocesamiento

- **Descripción:** El preprocesador expande macros, incluye headers (`#include`) y prepara el código para compilar.
- **Entrada:** `main.cpp` (código fuente → `.c`, `.cpp`, `.h`)
- **Salida:** `main.i` (código fuente expandido → `.i`, `.ii`)
- **Comando:** `g++ -E main.cpp -o main.i`

2 Compilación

- **Descripción:** Traduce el código fuente en instrucciones de ensamblador.
- **Entrada:** `main.i` (código fuente preprocesado)
- **Salida:** `main.s` (código ensamblador → `.s`, texto legible por humanos)
- **Comando:** `g++ -S main.i -o main.s`

3 Ensamblado

- **Descripción:** Convierte el ensamblador en código objeto, binario pero no ejecutable.
- **Entrada:** `main.s` (código ensamblador)
- **Salida:** `main.o` (código objeto, binario, no ejecutable)
- **Comando:** `g++ -c main.s -o main.o`

4 Enlazado (Linker)

- **Descripción:** Combina todos los objetos y bibliotecas en un único ejecutable que puede correr en la computadora. (Archivo final listo para correr en la CPU - Contiene código máquina completo)
- **Entrada:** `main.o` (código objeto) + Bibliotecas
- **Salida:** `main` (archivo ejecutable, código máquina listo)
- **Comando:** `g++ main.o -o main`

NOTA: Es importante aclarar que cada vez que se ejecute un comando correspondiente a una etapa, este ejecutará tanto la etapa anterior como la etapa actual. **Es decir:**

Comando	Descripción
<code>g++ -E main.cpp -o main.i</code>	Preprocesamiento sin compilación
<code>g++ -S main.i -o main.s</code>	Preprocesamiento y Compilación sin Enlazado
<code>g++ -c main.s -o main.o</code>	Preprocesamiento, Compilación, Ensamblado sin Enlazado
<code>g++ main.o -o main</code>	Preprocesamiento, Compilación, Ensamblado y Enlazado (Aquí también de enlazan las Bibliotecas)

🔗 Otras opciones más comunes:

- **-ansi**: en C es equivalente a **-std=c90**.
- **-Wall**: habilita todas las advertencias (warnings).
- **-Werror**: convierte las advertencias en errores.
- **-pedantic**: genera advertencias si se utiliza alguna extensión de GNU.
- **-std**: puede ser **-std=c90**, **-std=c99**, etc.

! ! ! Importante:

g++ (al igual que **gcc**) permite abreviar el proceso de compilación, ya que cada etapa ejecuta implícitamente las etapas anteriores.

Por este motivo, es posible invocar directamente una fase determinada sin ejecutar manualmente todas las anteriores.

```
$ g++ -S main.cpp MiClase.cpp
$ g++ -c main.cpp MiClase.cpp
$ g++ MiClase.cpp main.cpp -o app
```

En estos casos, el compilador ejecuta internamente las etapas previas necesarias, pero solo deja visible el archivo correspondiente a la etapa final solicitada:

- **-S** → genera archivos ensamblador (**.s**)
- **-c** → genera archivos objeto (**.o**)
- **sin flags** → genera el ejecutable final

Los archivos intermedios generados durante el proceso no se conservan por defecto.

🔍 Visualización de archivos intermedios

Si se desea **conservar y examinar los archivos intermedios** (preprocesado, ensamblador y objeto), se recomienda utilizar la opción **--save-temps**, que instruye al compilador a no eliminar dichos archivos:

```
$ g++ -S --save-temps main.cpp MiClase.cpp
$ g++ -c --save-temps main.cpp MiClase.cpp
$ g++ --save-temps MiClase.cpp main.cpp -o app
```

Esto permitirá observar explícitamente los archivos generados en cada etapa, como:

- **.ii** (preprocesado en C++)
- **.s** (ensamblador)
- **.o** (objeto)



Bibliotecas

Una **biblioteca (library)** es un conjunto de funciones ya compiladas que pueden reutilizarse en distintos programas sin reescribir el código.

No son código fuente (aunque pueden distribuirse junto a headers), **Son código máquina**.

1 Bibliotecas Estáticas (.a)

Son archivos que contienen código objeto ya compilado (**código de máquina**) y que se incorporan al ejecutable en el momento del enlazado.

Extensiones:

- Linux → **.a**
- Windows (MinGW) → **.a**
- Windows (MSVC) → **.lib**



Cómo funcionan

Durante el linkeo, el enlazador copia dentro del ejecutable las funciones necesarias desde la biblioteca.

Ejemplo:

```
$ g++ -static -Wall calc.cpp /usr/lib/x86_64-linux-gnu/libm.a -o calc_static
```

Si libm.a es estática, su código se integra dentro de app.

Explicación:

- **-static** → fuerza enlazado estático
- **-Wall** → activa advertencias
- **calc.c** → archivo fuente
- **-L** → directorio donde buscar bibliotecas
- **-lm** → enlaza contra **libm.a**
- **-o calc_static** → nombre del ejecutable



Resultado

- El ejecutable contiene todo el código.
- No depende de archivos externos para funcionar.
- Es más grande.

2 Bibliotecas Dinámicas (.so / .dll)

Son bibliotecas que no se integran completamente en el ejecutable, sino que se cargan en tiempo de ejecución.

Extensiones:

- Linux → `.so`
- Windows → `.dll`
- macOS → `.dylib`

⚙️ Cómo funcionan

En el enlazado solo se guarda una referencia a la biblioteca.

Al ejecutar el programa:

```
$ ./app
```

El sistema operativo carga la biblioteca dinámica en memoria.

Comando para enlazar una librería dinámica:

```
$ g++ -Wall calc.cpp -lm -o calc_dynamic
```

Explicación:

- `-Wall` → activa advertencias
- `calc.cpp` → archivo fuente C++
- `-lm` → enlaza la biblioteca matemática (libm)
- `-o calc_dynamic` → nombre del ejecutable

🔗 **NOTA:** 🔗 Comando `ldd` muestra las bibliotecas dinámicas (`.so`) de las que depende un ejecutable.

```
$ ldd nombre_del_ejecutable
$ ldd calc_dynamic
linux-vdso.so.1 (0x00007fffd5d5d000)
libm.so.6 => /usr/lib/x86_64-linux-gnu/libm.so.6 (0x00007f2c...)
libstdc++.so.6 => /usr/lib/x86_64-linux-gnu/libstdc++.so.6 (0x00007f2c...)
libc.so.6 => /usr/lib/x86_64-linux-gnu/libc.so.6 (0x00007f2c...)
/lib64/ld-linux-x86-64.so.2 (0x00007f2c...)
```

- `ldd` → Muestra qué bibliotecas dinámicas se cargarán en **tiempo de ejecución**.
- No muestra bibliotecas estáticas. **Solo aplica a binarios enlazados dinámicamente.**
- Indica la **ruta real donde se encuentran**.
- Sirve para depurar errores tipo: `library not found`. (Si aparece not found, la biblioteca falta en el sistema).

🔗 Resultado

- Ejecutable más pequeño.
- Puede compartirse la biblioteca entre varios programas.
- Requiere que la biblioteca esté instalada en el sistema.

Resumen

- **Bibliotecas = código ya compilado reutilizable.**
-  **Pertenecen a la etapa de ENLAZADO (Linking).**
 - **Biblioteca Estáticas** → se copian dentro del ejecutable.
 - **Biblioteca Dinámicas** → se cargan en tiempo de ejecución.
- **Son código máquina, no código fuente.**
 - Código objeto / código máquina compilado
 - Contienen símbolos y secciones binarias
 - Una **.a** es un archivo contenedor de múltiples **.o**.
 - Una **.so** es un binario enlazable dinámicamente.

[Índice \(←\)](#)